

EJPT - Section 1: Information Gathering



Elías Vallina
Spring 2026, Galway, Ireland

INFORMATION GATHERING

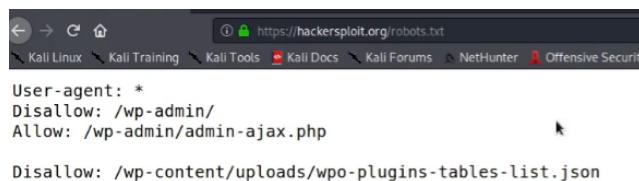
Information gathering and reconnaissance, they're essentially referring to the same. Footprinting as well but focuses more in a particular target.

Apache es un servidor web: cuando escribes una URL en el navegador, tu ordenador envía una petición y Apache la recibe y te devuelve la página (HTML, imágenes, etc.). Es como un restaurante: el navegador pide y Apache sería la cocina que prepara y te sirve lo que has pedido.

El servidor (MX) es por dominio, no por cada dirección : El registro MX de hackersploit.org dice: "para cualquier correo @hackersploit.org (info@hackersploit.org, elias@hackersploit.org, ...), entrégalo a este/estos hostnames".

MX significa Mail eXchanger: es un tipo de registro DNS que le dice a Internet a qué servidor(es) hay que entregar los correos de un dominio.

```
kali@kali:~$ whatis host
host (1) - DNS lookup utility
kali@kali:~$ host hackersploit.org
hackersploit.org has address 104.21.44.180
hackersploit.org has address 172.67.202.99
hackersploit.org has IPv6 address 2606:4700:3036::ac43:ca63
hackersploit.org has IPv6 address 2606:4700:3031::6815:2cb4
hackersploit.org mail is handled by 0 _dc-mx.2c2a3526b376.hackersploit.org.
```



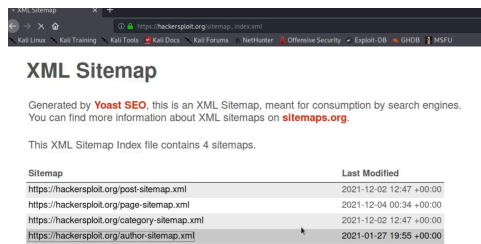
```
User-agent: *
Disallow: /wp-admin/
Allow: /wp-admin/admin-ajax.php

Disallow: /wp-content/uploads/wpo-plugins-tables-list.json
```

Eso es el robots.txt de la web: un archivo que **se encuentra en el root (/) del web server** que indica a los bots (Google, Bing, etc.) qué rutas pueden o no pueden rastrear.

- User-agent: * → reglas para todos los bots.
- Disallow: /wp-admin/ → no rastrear el panel de admin de WordPress.
- Allow: /wp-admin/admin-ajax.php → excepción: sí se permite ese archivo (lo usan funciones AJAX de WP).
- Disallow: /wp-content/uploads/wpo-plugins-tables-list.json → no rastrear/mostrar ese JSON.

Punto clave: no es seguridad. Solo "peticiones" para bots; cualquiera puede entrar a esas URLs si son públicas.

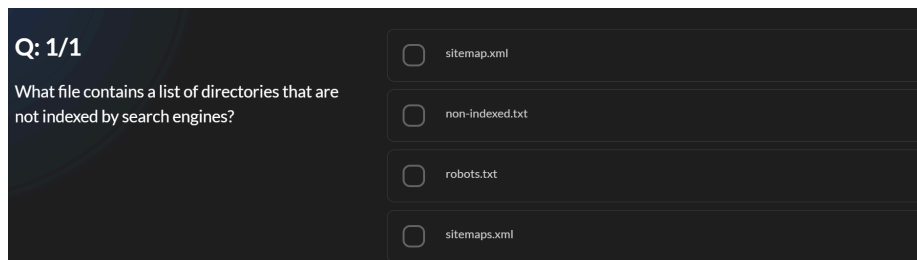


Adds-ons:

1- BuiltWithweb

2- terminal : whatweb hackersploit.org

3- webhttrack (app descargada).



Respuesta: [robots.txt](#).

Directorio / página web indexado por google o bing = google o bing incluyen ese directorio o página web en su índice (base de datos). Por lo que si no está indexado, al buscar en google esa pagina, NO saldrá entre los resultados. A menos que sepamos la URL exacta: si la ponemos en la barra de arriba de urls de google o bing, entonces sí que nos llevará a esa página.

Cuando creamos una página o hacemos cambios en esta, google si la rastrea significa que visitara su URL para ver su contenido, y decidirá si lo indexa o no según ciertos criterios:

Puede acceder a ella y no tiene noindex (es una instrucción para los buscadores que significa:“No metas esta página en Google” (no la muestres en resultados)) (ni errores 404/500 ni bloqueo/login). Tiene contenido útil y no duplicado (o es la canónica).

Parece importante (bien enlazada dentro de la web y/o con enlaces externos).

Es segura y de calidad (sin spam/malware).

La web funciona bien (no lenta ni llena de fallos).

Los bots (como Googlebot) :

Un bot (también llamado crawler o spider) es un programa automático que navega por Internet como si fuera un usuario, pero sin persona detrás.

Los bots de buscadores (por ejemplo Googlebot de Google o Bingbot de Bing) hacen esto:

1. Entran en una web
2. Leen la página (HTML/texto)
3. Siguen enlaces a otras páginas
4. Guardan lo que encuentran para poder mostrarlo en el buscador (indexarlo)

No “rastrean todas las páginas del mundo” porque es imposible; rastrean las que descubren (por enlaces, sitemaps, URLs conocidas) y priorizan según importancia, frecuencia de cambios, etc.

Y ojo: también hay otros bots no de buscadores (scrapers, bots maliciosos, etc.) que no siempre respetan robots.txt.

WHOIS ENUMERATION

whois da información de esa página web, como los nameservers, entre otras cosas. Nameservers = servidores que traducen un nombre de dominio a una dirección IP.

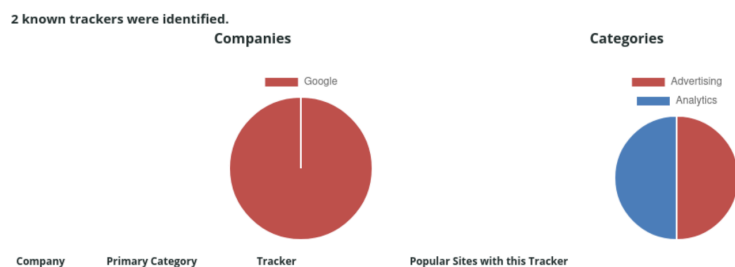
```
Information on how to contact the Registrant, Admin, or Tech contact of the domain:
kali@kali:~$ whois zonetransfer.me
Domain Name: ZONETRANSFER.ME
Registry Domain ID: D108500000003513097-AGRS
Registrar WHOIS Server:
Registrar URL: http://www.meshdigital.com
Updated Date: 2022-01-05T10:14:50Z
Creation Date: 2011-12-27T15:34:08Z
Registry Expiry Date: 2023-12-27T15:34:08Z
Registrar Registration Expiration Date:
Registrar: Mesh Digital Limited
Registrar IANA ID: 1390
Registrar Abuse Contact Email:
Registrar Abuse Contact Phone:
Reseller:
Domain Status: ok https://icann.org/epp#ok
Registrant Organization: DigiNinja
Registrant State/Province: Routrivill
```

But you typically gain information like

Website Footprinting With Netcraft

Página web para sacar info de una página web determinada

<https://sitereport.netcraft.com/>



Un tracker es una “herramienta de seguimiento” que una web mete dentro de la página (normalmente un script) para saber quién entra y/o para mostrar publicidad. Cuando tú visitas una web, tu navegador no solo habla con esa web, también puede enviar datos a empresas externas (por ejemplo Google) gracias a esos trackers.

Ejemplo: entras en sport.es.

- Con un tracker de analytics, sport.es puede saber cuánta gente ha leído una noticia, cuánto tiempo se queda y desde qué móvil/PC, para ver qué contenido funciona.
- Con un tracker de publicidad, sport.es puede mostrar banners y, si tú miras una noticia de fútbol, después en otras webs te pueden salir anuncios relacionados (remarketing).

En resumen: sirven para estadísticas (mejorar la web) y para publicidad (anuncios y seguimiento entre webs).

DNS Recon

Super importante, recordatorio:

google.com no es un servidor: es un nombre de dominio (un “nombre”), que se usa para llegar a un servicio web.

La conversión es así:

- google.com (nombre / dominio) → DNS lo traduce a → una IP
- Esa IP es la dirección de un servidor (o más exactamente, de un equipo/servicio de Google o de su infraestructura) que puede atenderte y mostrarte la web.

Así que:

- ❌ La IP no es “la IP de la página web” (porque una web no es un aparato).
- ✅ La IP es la dirección del servidor (o del balanceador/proxy) que sirve esa web.

detalle importante:

- Un dominio puede tener muchas IPs (Google usa muchas).
- Y una IP puede servir muchos dominios (varias webs en el mismo servidor).

En una sola frase:

DNS traduce el nombre de la web a la dirección (IP) del servidor que te va a responder.

dnsrecon -d zonetransfer.me

Al hacerlo, fijándonos en esto:

```
SOA nsztml.digi.ninja 81.4.108.41
```

SOA: el registro DNS “Start Of Authority”, indica quién es el servidor DNS principal de la zona del dominio; Cuando se escribe un dominio, por ejemplo:

zonetransfer.me: nuestro PC **no sabe** la IP. Entonces hace esto:

1. Nuestro PC pregunta a su DNS habitual (el de tu Wi-Fi / Google / Cloudflare):
“¿Cuál es la IP de www.zonetransfer.me?”
2. Ese DNS, si no la tiene en caché, busca quién manda para ese dominio.
En los **NS** del dominio pone que los “oficiales” son, por ejemplo:
nsztml.digi.ninja y nsztml2.digi.ninja
3. Entonces **ese DNS** le pregunta al servidor oficial:
“Oye nsztml.digi.ninja (IP **81.4.108.41**), ¿cuál es la IP de www.zonetransfer.me?”

Un **nameserver** es un servidor DNS que guarda la “agenda” de un dominio y responde preguntas como: “¿qué IP tiene sport.es?”. Cuando escribimos sport.es, mi PC pregunta al DNS y un nameserver le devuelve la IP para que el navegador pueda conectarse.

Un dominio suele tener varios nameservers. El primario es la fuente principal de la información DNS del dominio; los secundarios son copias que responden igual para respaldo y disponibilidad (si uno cae, otro sigue funcionando):

Más detalladamente: SOA vs NS

1. Identifica al "Líder": Mientras que el NS lista a todos los servidores (pueden ser 4 iguales), el SOA señala cuál de todos ellos es el servidor maestro (el que tiene la palabra final).
2. Controla las Versiones (Número de Serie): El SOA tiene un número (ej. 2026012201). Cada vez que haces un cambio en tu DNS (como mover mi pagina web de un servidor a otro, cambias el Registro A (la dirección IP). Cambio de Correo: Si contrato Google Workspace o Outlook, cambian los Registros MX.) , este número sube. El NS no sabe si algo cambió; el SOA es el que avisa: "¡Hay una nueva versión, actualícense!".

```
MX ALT1.ASPMX.L.GOOGLE.COM 142.251.9.26
MX ASPMX4.GOOGLEMAIL.COM 74.125.200.26
MX ASPMX2.GOOGLEMAIL.COM 142.250.150.27
```

MX es un servidor "la calle, dirección" del dominio: le dice a Internet dónde tiene que entregar los emails (el paquete) .

En la captura :MX ALT1.ASPMX.L.GOOGLE.COM ...; MX ASPMX.L.GOOGLEMAIL.COM ... Son servidores.

Eso significa: "Los emails para @ese-dominio los gestiona Google (aunque sea upc.edu o ub.edu, ...) (Gmail/Google Workspace)".

¿Y por qué hay varios? Para respaldo y prioridad: si el principal falla, se usa otro.

Las IPs que vemos al final (tipo 74.125... / 142.250...) son las IPs de esos servidores de correo.

Ej. rapido:

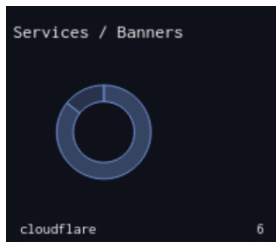
Envío un correo a suporte@empresa.com.

1. El servidor de correo del que envía pregunta al DNS:
"¿Cuáles son los MX de empresa.com?"
2. DNS responde: "entrega el correo al servidor MX: ASPMX.L.GOOGLE.COM (prioridad 1)"
3. Ese servidor (de Google) recibe el email y lo mete en la bandeja del usuario (Google Workspace/Gmail).

Alternativa a [dnsrecon -d zonetransfer.me](https://dnsrecon-d.zonetransfer.me) :

dnsdumpster.com

Al entrar y poner hackersploit.org, vemos:



Cuando una web usa **Cloudflare**, tú normalmente no hablas con el servidor real de la web. Hablas con Cloudflare, y Cloudflare reenvía la petición al servidor “origin” (real) y te devuelve la respuesta. Eso es exactamente el concepto de reverse proxy.

La diferencia con el “proxy” típico que imaginas

- Proxy normal (forward proxy): lo usas tú para salir a Internet.
Ej: tú → proxy → web (lo configuras en tu navegador/empresa)
- **Cloudflare (reverse proxy): lo pone el dueño de la web delante de su servidor.**
Ej: tú → Cloudflare → servidor real (lo configura el dueño del sitio)

Qué añade Cloudflare además de proxy

- CDN/cache (más rápido)
- WAF/firewall (bloquea bots/ataques)
- Protección DDoS
- Ocultar IP real del servidor muchas veces

Resumen: sí, Cloudflare actúa como proxy, concretamente un reverse proxy (un “proxy del servidor”, no del cliente).

WAF With wafwoof

Un WAF (Web Application Firewall) es un “firewall” pensado específicamente para webs (HTTP/HTTPS).

- Firewall normal = el portero mira tu DNI y si tienes entrada (IP/puerto).
- WAF = además revisa qué llevas en la mochila (contenido HTTP) y si llevas algo raro, te para.

wafwoof <https://hackersploit.org>

```
-(kali@kali)-[~]
$ wafw00f hackertube.net

?
??
( ) ;
/ /

~ WAFW00F : v2.3.1 ~
~ Sniffing Web Application Firewalls since 2014 ~

+] Checking https://hackertube.net
+] The site https://hackertube.net is behind Cloudflare (Cloudflare Inc.) WAF.
-! Number of requests: 2
```

wafwoof <https://hackersploit.org> **-a** : -a, --findall: Find all WAFs, do not stop testing on the first one. (que no se pare en el primer WAF q detecte, q muestre todos).

SUBDOMAIN ENUMERATION WITH Sublist3r

El siguiente comando sirve para encontrar subdominios que **no estan indexados** por los engines como google o yahoo, pero que alguna vez si lo fueron y hay alguna manera de hallarlos. Este es el comando que lo intenta encontrar.

sublist3r -d hackersploit.org -e google,yahoo (e es de engine y d para q pongas el domain).

Ejemplo típico en casa de como funciona ips publicas y privadas

- IP pública (la ve Internet): 86.12.34.56 → solo hay una para todos los usuarios conectados a ese router.
- IPs privadas (solo dentro de tu Wi-Fi, no son visibles por los demas fuera del alcance del router): Mi portátil: 192.168.1.10; mi móvil: 192.168.1.11, etc..

Google Dorks

site:ine.com → limita resultados a ine.com (normalmente incluye subdominios).

inurl:forum → solo resultados cuya URL contiene forum.

site:ine.com inurl:forum → dentro de ine.com, URLs que contengan forum.

site:*.ine.com → se usa para “subdominios”, pero en Google es poco fiable / no oficial; mejor site:ine.com o site:sub.ine.com.

intitle:admin → páginas cuyo título contiene admin.

inurl:admin → páginas cuya URL contiene admin.

- Nota: intitle (mal escrito) no funciona como operador.

filetype:xlsx → resultados que son archivos Excel .xlsx (descargables si no hay login).

site:ine.com instructors → dentro de ine.com, páginas que contengan la palabra instructors en cualquier parte (texto/título/URL).

- Si lo quieres en URL: inurl:instructors; en título: intitle:instructors.

cache:ine.com → intenta mostrar la copia en caché de Google de esa URL (si existe; muchas veces no).

Wayback Machine (escribirlo en google) → no es comando de Google: es el archivo histórico de archive.org para ver versiones antiguas de una web.

inurl:passwd.txt → busca URLs que contengan passwd.txt (suele apuntar a archivos sensibles expuestos; úsalo solo en tu propio dominio/lab).

exploit-db.com/google-hacking-database : Lista de *dorks* (búsquedas avanzadas) para **OSINT/auditoría**: encontrar archivos expuestos, paneles, “index of”, etc. (mejor para tu propio dominio/lab).

Email Harvesting with theHarvester

```
(kali@kali)-[~]  
$ theHarvester -d INE -b duckduckgo
```

Esto lo que hace es ejecuta theHarvester para hacer **OSINT (recopilar y analizar datos que ya están accesibles sin hackear nada)**: busca en DuckDuckGo resultados públicos relacionados con “INE” y extrae de ahí información como emails y hosts/subdominios que aparezcan **indexados (que esten en ese navegador de forma accesible)**.

- **-d INE**: objetivo/término (idealmente sería un **dominio** real, tipo **ine.com**, para afinar).
- **-b duckduckgo**: fuente de búsqueda (**DuckDuckGo**).

Mejor poner más buscadores en el comando para tener mejores resultados:

```
s3rpent@stronghold ~ $ theHarvester -d INE -b duckduckgo,baidu,bing,yahoo
```

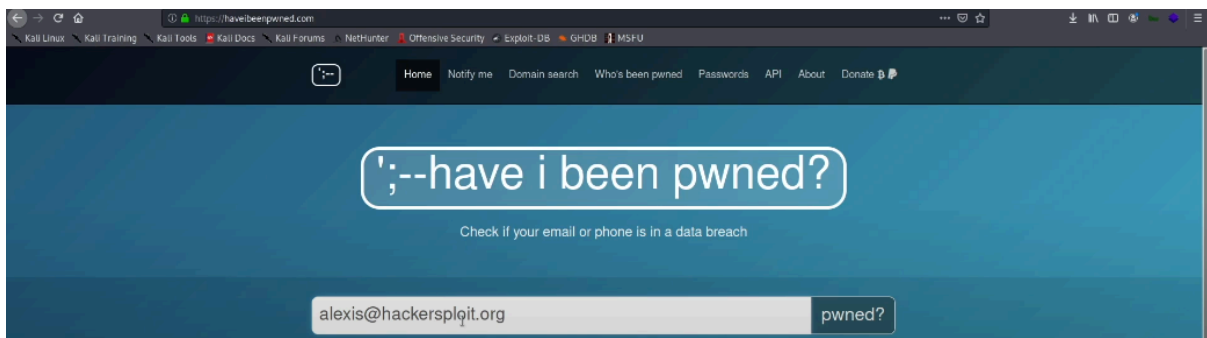
Casi todas las pruebas q ha hecho no le han dado resultados, menos esta q le ha dado 17 resultados: Esta parece interesante poner ine.com y esos buscadores en concreto.

```
s3rpent@stronghold ~ $ theHarvester -d ine.com -b duckduckgo,baidu,bing,yahoo,urlscan
```

```
[*] Hosts found: 17  
-----  
3dget.ine.com  
auth.ine.com  
certs.ine.com  
checkout.ine.com  
courses.assets.ine.com  
courses.ine.com  
dashboard.ine.com  
get.ine.com  
info.ine.com  
learn.ine.com  
legacy-community.ine.com  
members.ine.com  
my.ine.com
```

Luego ha hecho este otro comando y da una locura de resultados, muchos mas que el anterior!

```
s3rpent@stronghold ~ $ theHarvester -d ine.com -b duckduckgo,baidu,bing,yahoo,urlscan,crtsh,intel  
x,otx,rapiddns,whoisxml,tomba
```



página muy buena para ver si esa pagina tiene alguna brecha 🙌

Active Information Gathering

DNS Zone Transfers

DNS Records

- + A - Resolves a hostname or domain to an IPv4 address.
- + AAAA - Resolves a hostname or domain to an IPv6 address.
- + NS - Reference to the domains nameserver.
- + MX - Resolves a domain to a mail server.
- + CNAME - Used for domain aliases.
- + TXT - Text record.
- + HINFO - Host information.
- + SOA - Domain authority.
- + SRV - Service records.
- + PTR - Resolves an IP address to a hostname

```
li@kali:~$ sudo vim /etc/hosts
```

Al escribir este comando se abre esto:

```
127.0.0.1    localhost
127.0.1.1    kali

# The following lines are desirable for IPv6 capable hosts
::1          localhost ip6-localhost ip6-loopback
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

192.168.1.1  router.admin
```

si me fijo en la ultima línea, podemos poner una direccion IP y su nombre asociado que queremos y entonces para entrar al router en vez de poner en google los numeritos 192.168.1.1 pongo router.admin y ya lo entiende (hace el la conversión al numero).

Txt: 1. El "Libro de Registro" Oficial (DNS)

Imagina que Internet tiene un libro de registro de propiedades (el sistema DNS).

- **Solo el dueño legal de google.com puede escribir en la página de google.com de ese libro.**
- Si un estafador intenta poner un registro TXT para google.com, el "notario" (los servidores raíz de Internet) le dirá: **"Usted no es el dueño, no puede escribir aquí"**.
- Por eso, cuando mi PC pregunta "¿Cuál es el TXT de google.com?", la respuesta viene de una cadena de confianza que el estafador no puede manipular.

2. El "Reto" Secreto (Verificación)

Imagina que quieres contratar un servicio de publicidad.

1. **El servicio te dice:** "Para saber que eres el dueño de `minegocio.com`, pon este código único en tu TXT: `TOKEN_999`".
2. **El estafador:** No puede hacerlo porque no tiene acceso al panel de control de tu dominio.
3. **Tú:** Entrás con tu contraseña a tu proveedor (ej. GoDaddy) y lo pones.
4. **Conclusión:** Cuando el servicio lee `TOKEN_999` en el DNS público de tu dominio, sabe que **solo tú pudiste haberlo puesto**, porque solo tú tienes la llave de esa "página del libro".

3. El Candado Verde (Certificados SSL)

Para la navegación web diaria, el registro TXT no es lo que te asegura que estás en Google, sino el **Certificado SSL** (el candado de la barra de direcciones).

- Cuando entras a `google.com`, el servidor te muestra un certificado firmado por una **Autoridad de Certificación** (un tercero de confianza).
- Esa autoridad ha verificado físicamente y legalmente que esa empresa es Google Inc. antes de darle el certificado.
- Nuestro navegador (Chrome, Safari) tiene una lista de esas autoridades en las que confía. Si el estafador intenta imitarlo, tu navegador dirá: "**¡Peligro! Este certificado no es válido**".

Al instructor le gusta más este comando que el dns enum (`dnsenum zonetransfer.me`):

```
kali@kali:~$ dig axfr @nsztl1.digi.ninja zonetransfer.me
```

Tener en cuenta que puede de la lista de dominios que nos puedan dar estos 2 comandos (`dnsenum` y `dig`), haya direcciones web que no sean accesibles porque sean privadas o internas de la empresa (por ejemplo la vanguardia tendra direccionesweb q tendran el acceso denegado a los demas).

Trying Zone Transfers and getting Bind Versions:

```
Trying Zone Transfer for hackersploit.org on dee.ns.cloudflare.com ...  
AXFR record query failed: FORMERR
```

```
Trying Zone Transfer for hackersploit.org on jim.ns.cloudflare.com ...  
AXFR record query failed: FORMERR
```

Brute forcing with `/usr/share/dnsenum/dns.txt`:

Una transferencia de zona (AXFR) es cuando un servidor DNS “autoritativo” te entrega **toda la lista completa de registros DNS del dominio** (subdominios, A, MX, TXT, etc.) de golpe.

Pensar en ello como:

- Normal: “Oye DNS, dime la IP de www.hackersploit.org”
- AXFR: “Oye DNS, **pásame TODO el catálogo** del dominio hackersploit.org”

Por qué lo intenta dnsenum

Porque si funciona, es el método **más rápido y más potente** para sacar subdominios y registros sin adivinar nada.

Da error en la captura porque solo puede acceder dns secundarios autorizados por el primario. La consulta la he hecho yo, la IP de mi PC, desde mi kali, al ejecutar `dnsenum hackersploit.org`.

Lo que pasa es que **dnsenum** no “usa” un DNS secundario autorizado, la identidad de este no puede prestarse. Solo hace peticiones DNS normales **desde mi IP** a los nameservers autoritativos del dominio (en mi caso, los de Cloudflare).

O sea: **no es que pueda elegir hacerlo “a través” del secundario**. El secundario es una máquina concreta, con una IP concreta, **y el primario solo le da el AXFR a esa IP**.

Ahora centrándonos en el plan B: el brute forcing:

Como **AXFR falló**, ya no puede obtener la lista completa “gratis”.

Entonces hace lo típico: **adivinar subdominios comunes** y comprobarlos uno a uno.

El archivo que veo:

`/usr/share/dnsenum/dns.txt`

es simplemente **la wordlist** (lista de posibles nombres).

Y para cada uno hace consultas tipo:

- “¿Existe un registro A/AAAA/CNAME para api.hackersploit.org?”
- Si el DNS responde con algo válido → **lo apunta como encontrado**
- Si responde NXDOMAIN/no existe → **lo descarta**

```
kali@kali:~$ cat /usr/share/dnsenum/dns.txt
```

👉 `cat` aquí **solo muestra en pantalla el contenido** del archivo.

En la captura de encima, lo que se hace es:

- abre el fichero `dns.txt`

- imprime todas sus líneas en la terminal

Opción ya que no funciona axfr:

```
kali@kali:~$ fierce -dns zonetransfer.me
```

HOST DISCOVERY WITH NMAP

Primer comando interesante:

```
kali@kali:~$ ip a s
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 08:00:27:5c:65:26 brd ff:ff:ff:ff:ff:ff
    inet 192.168.2.190/24 brd 192.168.2.255 scope global dynamic noprefixroute eth0
        valid_lft 35673sec preferred_lft 35673sec
    inet6 fe80::a00:27ff:fe5c:6526/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
kali@kali:~$
```

Comando importantísimo:

```
(kali@kali)-[~]
$ sudo nmap -sn 192.168.238.0/24
```

con sn:

- No hace port scan
- Sale rápido

sin sn :

- host discovery
- escanea puertos (si responde alguien cuando hacemos host discovery).

nmap por defecto escanea solo los 1000 puertos mas comunes.

```
kali@kali:~$ sudo netdiscover -i eth0 -r 192.168.2.0/24
```

Diferencia:

netdiscover : descubrimiento por ARP

nmap -sn : descubrimiento “multi-método” (ARP/ICMP/TCP según entorno).

```
root@attackdefense:~# nmap -Pn 10.4.19.218
```

mejor poner el -Pn porque a veces da error.

PORT	STATE	SERVICE
80/tcp	open	http
135/tcp	open	msrpc
139/tcp	open	netbios-ssn
445/tcp	open	microsoft-ds
3389/tcp	open	ms-wbt-server
49154/tcp	open	unknown
49155/tcp	open	unknown

En la captura de encima: ¿80 = HTTP “de verdad”?

Probablemente sí, pero con lo que se ve ahí, nmap está diciendo: “el puerto 80 está abierto y parece ser HTTP” (por convención de que el 80 se usa la mayoría de veces para http).

```
root@attackdefense:~# nmap -Pn -p 80,445,3389 10.4.19.218
```

Así especifico qué puerto/s me interesa mirar en concreto.

En la siguiente imagen, el estado filtered no significa que esta cerrado ese puerto, es solo una posibilidad. otra posibilidad seria que hay un firewall que esta filtrando el trafico.

PORT	STATE	SERVICE
8080/tcp	filtered	http-proxy

En la siguiente imagen, vemos que se escanearan los puertos del 1 al 1000

```
root@attackdefense:~# nmap -Pn -p1-1000 10.4.19.218
```

Si queremos hacer en modo Fast (escanea solo los 100 puertos mas importantes):

```
root@attackdefense:~# nmap -Pn -F 10.4.19.218
```

Nmap por defecto realizara un TCP port scan. Si queremos UDP scan hay que especificarlo. Se hace basicamente poniendo -sU:

```
root@attackdefense:~# nmap -Pn -sU 10.4.19.218
```

Si queremos unos resultados mas detallados, ponemos -v (verbose):

```
root@attackdefense:~# nmap -Pn -F 10.4.19.218 -v
```

```
root@attackdefense:~# nmap -Pn -F -sV 10.4.19.218
```

👉 con -sV conseguimos **detectar servicios y versiones**.

Después de ver qué puertos están open, nmap:

- se conecta a esos puertos
- envía “sondas” típicas (por ejemplo, una petición HTTP, un saludo SMTP, etc.)
- lee la respuesta (banner / comportamiento) y con eso intenta identificar:
- qué servicio real está corriendo (HTTP, SMB, SSH, FTP...)
- qué software es (Apache, nginx, IIS, OpenSSH...)
- y a veces la versión (por ejemplo “Apache 2.4.57”)

Nota: Puerto 80 abierto ≠ “seguro que es HTTP”

Lo más común es que sí sea HTTP, pero se puede usar para otras cosas o para “engañar”.

Ejemplo:

Sin -sV:

- 80/tcp open http (porque el puerto 80 suele ser HTTP)

Con -sV:

- 80/tcp open http Apache httpd 2.4.x

Con el -O , hacemos que nos de el operating system:

```
root@attackdefense:~# nmap -Pn -F -sV -O 10.4.19.218 -v
```

Si añadimos -sC, estamos diciendo: Además del escaneo, **ejecuta los scripts por defecto** (NSE default), que son mini-programas que vienen con Nmap y al ejecutarlos el objetivo es sacar más información de estos puertos abiertos:

```
root@attackdefense:~# nmap -Pn -F -sV -O -sC 10.4.19.218 -v
```

A continuación, depende de la TX que ponga, donde X es un número, hará una cosa u otra:

```
-T paranoid|sneaky|polite|normal|aggressive|insane (Set a timing template)
```

Ej: T0 = paranoid

Ej con T5:

```
root@attackdefense:~# nmap -Pn -F -T5 -sV -O -sC 10.4.19.218 -v
```

Si queremos guardar el resultado q nos ha salido por la terminal, en un documento, hacemos:

```
root@attackdefense:~# nmap -Pn -F 10.4.19.218 -oN test.txt
```

-oN test.txt le dice a **nmap** que **guarde el resultado del escaneo en un archivo** llamado test.txt en formato **normal** (legible como el que ves en pantalla).

- -o = output (salida)
- N = **Normal** format
- test.txt = nombre del fichero donde lo escribe

Si hacemos :

```
root@attackdefense:~# cat test.txt
```

Nos mostrará por pantalla básicamente lo que hay dentro del fichero este.

A continuación, este comando -oX test.xml = **output XML** a un archivo llamado test.xml.

```
root@attackdefense:~# nmap -Pn -F 10.4.19.218 -oX test.xml
```

Teoría importante DNS:

Cuando escribo **sport.es**:

1. **DNS** traduce "sport.es" a una **IP** (te dice a qué **servidor/máquina** tienes que ir). Puede ser máquina física o máquina virtual corriendo de una máquina física. Pensarlo como un superordenador que tiene guardado toda la info de sport.es
2. Nuestro ordenador se conecta a esa IP (máquina física) (normalmente por **HTTPS 443**). A veces primero pasas por un **CDN/reverse proxy**.
3. Una vez conectas, el **servidor web** (el programa que se encuentra dentro del servidor: Apache/Nginx/IIS) **recibe tu petición HTTP** y te **envía el HTML**.
4. Luego ese mismo servidor web (o el proxy) te manda también **CSS, JS e imágenes**, y el navegador lo renderiza y lo ves.

Clave:

- **Servidor** = la máquina/infraestructura detrás de sport.es
Servidor web = el software en esa máquina que te entrega la página web.

Clave 2:

Dominio (sport.es) = nombre

IP = dirección real de red

Una IP puede alojar muchas webs, no necesitas 10 IPs para 10 webs.

Cuando varias webs comparten la misma IP, tu navegador le dice al servidor qué web quieres con el campo Host en la petición HTTP.

Ejemplo: si pedimos sport.es manda Host: sport.es; si pides lavanguardia.com manda Host: lavanguardia.com.

El servidor lee ese Host y por eso sabe qué página devolver.

Wordpress es un software para hacer páginas web y no tener que programarlas desde cero, es una especie de plantilla.

ejemplo de **sport.es** hecho en WordPress:

Si entras en sport.es (WordPress), estos son los 3 directorios clave

1. /wp-admin/ → *Panel de administración*
 - Es donde entra el admin del sitio para publicar, cambiar cosas, gestionar usuarios/plugins, etc.
 - Ej: sport.es/wp-admin/
2. /wp-includes/ → *Núcleo interno de WordPress (core)*
 - Archivos “del motor”: funciones, librerías, JS/CSS del core.
 - Normalmente no se toca, viene con WordPress.
 - Ej: sport.es/wp-includes/
3. /wp-content/ → *Contenido “real” del sitio (lo que personalizan)*
 - Aquí viven:
 - themes/ (tema = diseño)
 - plugins/ (plugins = funciones extra)
 - uploads/ (imágenes, PDFs, etc.)
 - Ej: sport.es/wp-content/

Frase para recordarlo fácil:

- wp-admin = control
- wp-includes = motor
- wp-content = lo que ves y lo que suben

Estructura básica wordpress:

WordPress (raíz del sitio)

- |— .htaccess
- |— index.php
- |— license.txt
- |— readme.html
- |— wp-activate.php
- |— wp-admin/ ← Panel de administración
- |— wp-blog-header.php
- |— wp-comments-post.php
- |— wp-config.php ← ¡AQUÍ ESTÁ! (configuración principal)
- |— wp-content/ ← Lo más importante en recon
 - | |— plugins/ ← Plugins instalados
 - | |— themes/ ← Temas (el activo + otros)
 - | | |— twentyfour/ ← Ejemplo de tema (como el tuyo)
 - | | |— assets/ ← Fonts, images, etc.
 - | |— uploads/ ← Archivos subidos (flag 3 aquí)
 - | | |— 2024/
 - | | | |— 01/ ← Carpetas por año/mes
 - | | | |— ... ← Archivos sospechosos dentro
 - | | |— mu-plugins/ ← Plugins obligatorios (a veces)
- |— wp-cron.php
- |— wp-includes/ ← Archivos core de WP (no tocar)
- |— wp-links-opml.php
- |— wp-load.php
- |— wp-login.php
- |— wp-mail.php
- |— wp-settings.php
- |— wp-signup.php
- |— xmlrpc.php

Webroot (raíz web)

Es la carpeta del servidor desde la que Apache sirve archivos por Internet. Todo lo que puedas abrir con una URL tipo:

- `http://target.ine.local/`
- `http://target.ine.local/wp-content/...`
- `http://target.ine.local/robots.txt`

Si lo puedes pedir desde el navegador con `/algo`, está en el webroot (o dentro).

No es webroot

Son carpetas del sistema del servidor (físico) que no se pueden abrir por URL, como:

- `/etc/`
- `/var/log/`
- `/home/...`
- `/root/`
- `/var/lib/mysql/`

Si no puedes escribirlo en el navegador como `http://target.ine.local/...`, no es webroot.

Importante: Una carpeta en tu PC llamada `target.ine.local` (por ejemplo descargada con HTTrack) no es el webroot real, es solo una copia local.

IMPORTANTISIMO

- **Servidor (máquina/SO):** donde existen carpetas como `/etc`, `/var/log`, `/root`, `/home`. Son del sistema y servicios internos.
- **Servidor web (Apache/Nginx):** es un programa que corre en esa máquina y solo sirve lo que esté dentro del webroot (`DocumentRoot`), por ejemplo `/var/www/html` o la ruta que tenga configurada.

Así que: `/etc`, `/var/log`, etc. = carpetas del servidor (SO).

El servidor web normalmente no las expone por URL (a menos que esté mal configurado o haya un fallo).

CURL

Sirve para hacer peticiones a una URL (como si fuera un navegador) y ver la respuesta del servidor.

En labs se usa mucho porque deja ver cosas “técnicas” rápido: headers, redirecciones, cookies, etc.

Por lo tanto, al hacer `curl`, el terminal te muestra **la respuesta del servidor** (lo que un navegador recibiría “por dentro”). De esta manera, el navegador al yo escribir sport.es, basicamente recibe del servidor la respuesta en texto con :

Headers: metadatos (tipo de contenido, cookies, caché, servidor, etc.)

Body: el contenido visual (HTML/JSON/imagen/etc.)

LAB EXAM footprinting:

FLAG 1: **The server proudly announces its identity in every response. Look closely; you might find something unusual.**

“Identidad” aquí no es el dominio ni la IP: es cómo se identifica el servidor en los headers, típico en el header : nos da el `webserver`: (Apache/Nginx/IIS, a veces con versión), es decir, el servidor web que nos está atendiendo al conectarnos a `target.ine.local`.

Con `curl http://...` **solo estaba viendo solo el body/HTML** pero para este flag necesitas ver headers:

- `curl -I http://target.ine.local/` (solo headers)
- `curl -I -L http://target.ine.local/` (si hay redirecciones)

“Atendiendo” = el software/servicio (Nginx/Apache) que recibe tu petición y responde.

